Politechnika Wrocławska

Struktury danych

Analiza porównawcza wydajności list wiązanych i tablic dynamicznych w języku C++

Piotr Frąc

Marzec 2026

1 Wprowadzenie

Projekt skupia się na analizie porównawczej dwóch fundamentalnych struktur liniowych: *linked list* i *dynamic array*
Celem pracy jest implementacja obu struktur w języku C++ oraz przeprowadzenie testów wydajnościowych dla podstawowych operacji, takich jak:

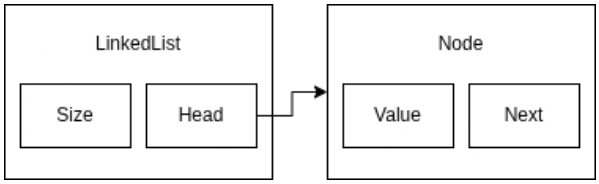
- dodawanie i usuwanie elementów (na początku, na końcu oraz na zadanym indeksie),
- wyszukiwanie elementów o określonej wartości,
- dostęp do elementu poprzez indeks.

Badanie ma na celu wykazanie różnic w złożoności obliczeniowej oraz praktycznym czasie wykonania, wynikających ze specyfiki alokacji pamięci
Pełny kod źródłowy projektu wraz z historią zmian dostępny jest w repozytorium pod adresem: `https://github.com/Elrior-programmer/Struktury-Danych/tree/main/P1%20-%20Lista_wizana`

1.1 Zaimplementowane struktury danych

Wszystkie struktury danych zostały wykonane zgodnie z wykładami i prezentacjami dr inż. Radosława Rudego, które są dostępne pod linkiem: `https://kam.pwr.edu.pl/jaroslaw-rudy/#tab-struktury-danych-wyklad`

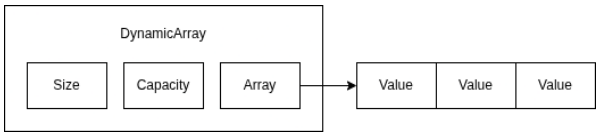
Lista wiązana



Zaimplementowana **lista** jest przykładem *Listy jednokierunkowej liniowej (Singly Linked List)*, która charakteryzuje sie:

- **Jednokierunkowość:** Każdy węzeł (*node*) przechowuje wskaźnik wyłącznie do swojego następcy. Oznacza to, że przeglądanie struktury możliwe jest tylko od głowy (*head*) w stronę końca listy.
- **Liniowość (brak cyliczności):** Struktura posiada wyraźnie zdefiniowany początek oraz koniec. Ostatni element przechowuje wartość pustą (`nullptr`), co stanowi jednoznacznie zakończenie sekwencji danych.

Tablca Dynamiczna



Zaimplementowana **tablica dynamiczna** charakteryzuje się specyficznymi usprawnieniami implementacyjnymi:

- **Inicjalizacja niezerowa:** Tablica rozpoczyna pracę z domyślną pojemnością $capacity = 4$, co eliminuje potrzebę realokacji przy pierwszych operacjach dodawania.
- **Progresja geometryczna ($2 \times capacity$):** W przypadku przepełnienia bufora, jego rozmiar jest zwiększany dwukrotnie. Zapewnia to zamortyzowany koszt dopisania elementu na poziomie $O(1)$.
- **Redukcja pamięci ($capacity/2$):** Gdy liczba elementów spadnie do poziomu $size \leq capacity/4$, rozmiar tablicy jest zmniejszany o połowę.

1.2 Wykorzystane dane

sprawdzam ile liniej do nowej strony
sprawdzam ile liniej do nowej strony